

SYSTÈMES D'INFORMATION
& GÉNIE LOGICIEL — ARLA

Expense.io : audit et refacto d'architecture

Équipe 1Pacte

SYNTHÈSE EXÉCUTIVE

L’essentiel du projet

L’application marchait, mais elle exposait nos clients et coûtait cher à faire évoluer. Elle est maintenant sûre et prête pour la suite, sans rien changer pour nos clients.

1

Le risque métier

Une entreprise cliente pouvait lire les **justificatifs d’une autre** : perte de confiance, données personnelles exposées, contrats B2B en jeu. **4 failles critiques** sur 46 constats.

2

Ce qu’on a fait

Une **isolation des clients garantie** par la base, une architecture **en modules** et des e-mails fiables même si le serveur mail tombe. **Aucun client à réécrire** : l’API ne change pas.

3

Ce que ça apporte

Risque critique éliminé : 12 failles critiques et élevées corrigées, prouvées par 620 tests. Des évolutions **plus rapides et moins risquées** : une fonctionnalité complète en un prompt.

POUR L’ENTREPRISE Vendable sans craindre la fuite, évolutif sans tout casser.

1 · CE QU'ON A TROUVÉ

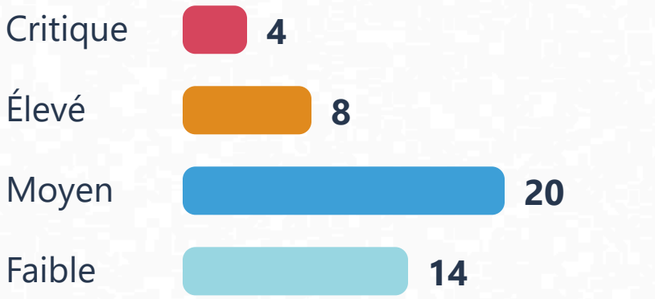
46 constats : l'isolation des clients d'abord

Classés par impact métier × facilité d'exploitation, tous rejoués dans l'app.

1 **Lire** l'existant

2 **Rejouer** l'app, rôle par rôle

3 **Figer** : +140 tests de caractérisation



8 élevés

Rôles usurpables, double décision acceptée, OCR qui sature le serveur, chemins serveur exposés.

Critique

Reçus d'une autre entreprise

Un manager ouvre le justificatif d'un salarié d'une autre entreprise.

Critique

Filet de la base désactivé

L'app se connecte en superuser : l'isolation par entreprise ne s'applique jamais.

Critique

Injection dans le tri des notes

`?sort=` exécute du SQL, avec les droits superuser.

Critique

Secret de session dans le code

Qui lit le dépôt peut fabriquer une session RH de n'importe quelle entreprise.

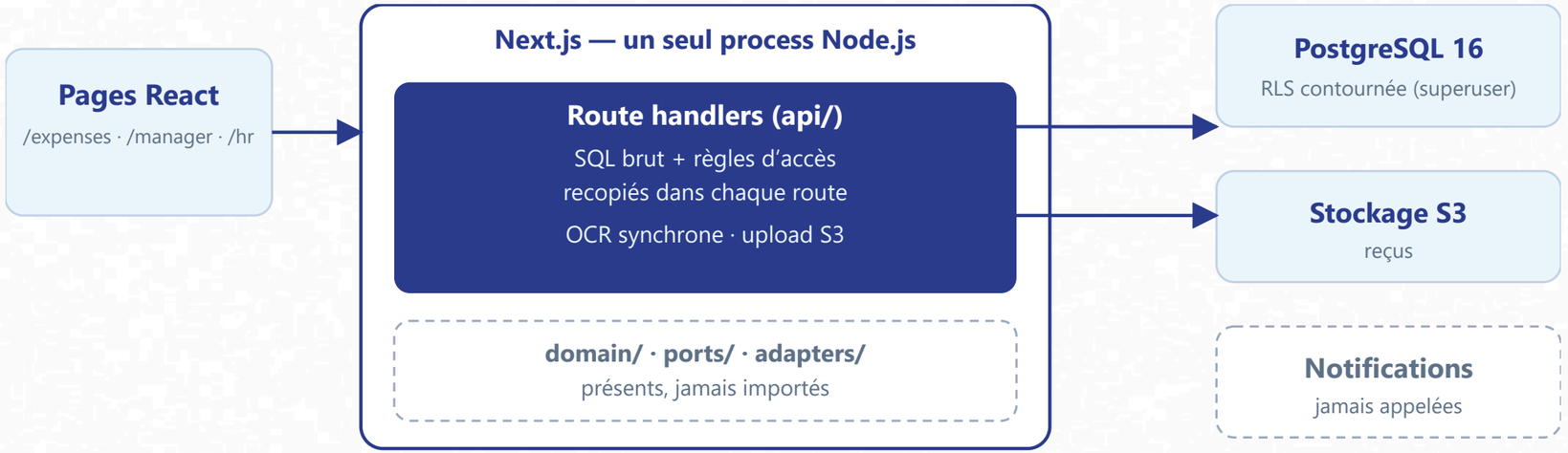
Et pour l'utilisateur : le salarié n'est **jamais prévenu** de la décision (e-mails codés, jamais envoyés), et la note d'un employé sans manager n'a **aucun validateur**.

POUR NOS CLIENTS Leurs justificatifs étaient lisibles par d'autres entreprises.

1 · CE QU'ON A TROUVÉ · ARCHITECTURE AS-IS

Un monolithe où tout se mélange

La couche hexagonale existait, mais personne ne l'importait.



7×
règles d'accès recopiées, une par route

0
filet dans la base : l'app s'y connecte en superuser

0
e-mail envoyé : les notifications ne sont jamais appelées

POUR NOS CLIENTS Une règle oubliée dans une route : un client voit les données d'un autre.

2 · CE QU'ON A DÉCIDÉ

Des frontières nettes, sans distribuer

Trois besoins tirés de l'audit : isoler les clients, ne rien casser chez eux, prévenir sans bloquer.

OPTION	APPORT	LIMITE
A · Monolithe en couches nettoyé	Léger.	Règles d'accès toujours éparpillées.
B · Monolithe modulaire hexagonal	Frontières nettes, métier testable.	Un serveur mail en panne bloque la validation.
C · B + file d'e-mails ✓	**Répond aux 3 besoins.**	**Un process de plus à exploiter (worker).**
D · Microservices	Déploiement indépendant.	Réseau et exploitation ×N, pour aucun besoin de charge.

Isoler : la base l'impose

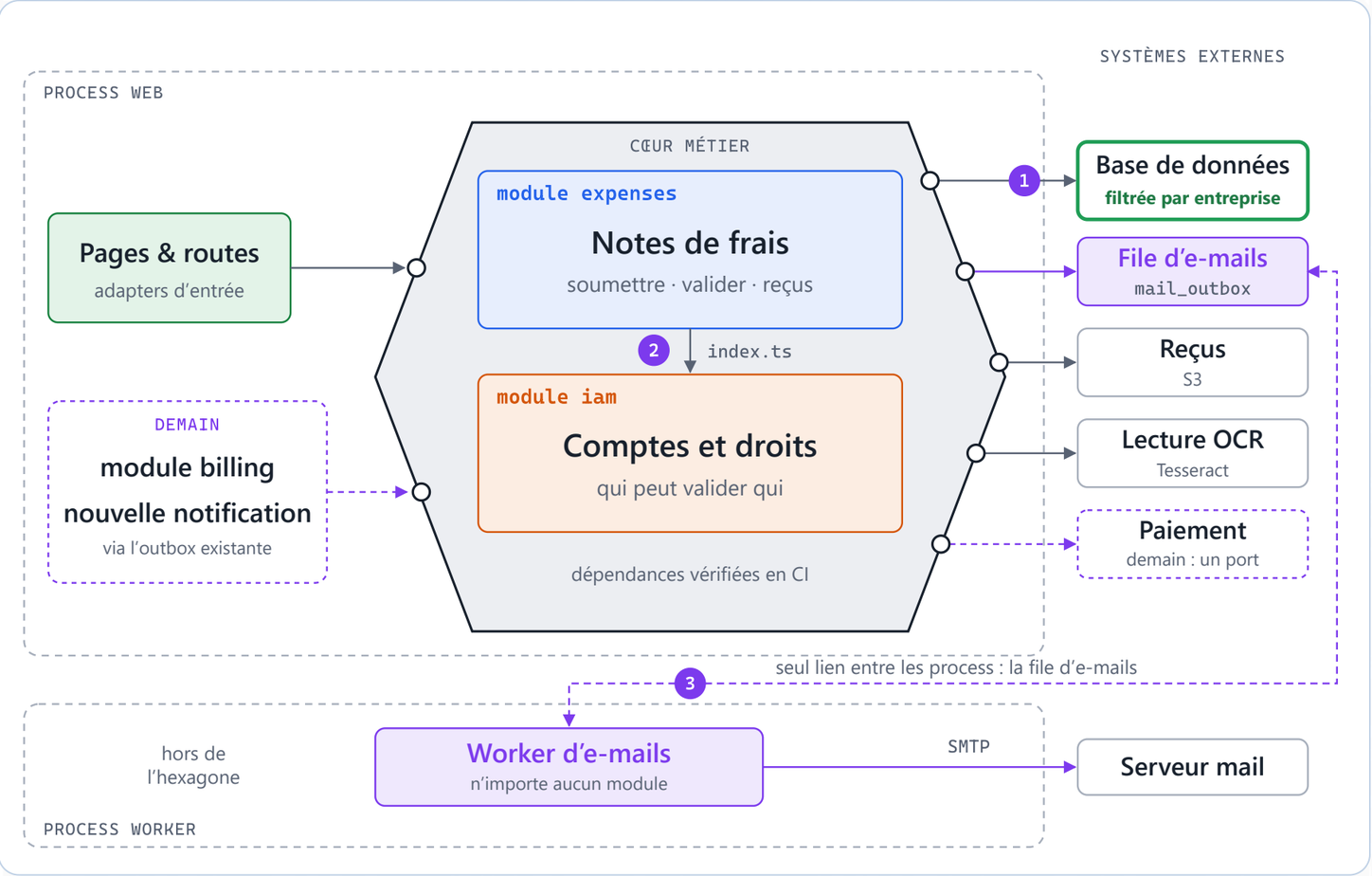
Ne rien casser : API figée

Charge : 360–560 req/s suffisent

Choix écrit : ADR 0002

POUR L'ENTREPRISE Une seule application à exploiter, découpée pour accueillir la suite.

Isolée par la base, découpée, fiable



1. **Isoler les clients.** Chaque requête passe par la base, qui ne rend que les données de l'entreprise de l'utilisateur. Un oubli dans le code ne fuit plus.
2. **Évoluer sans casser.** Un module n'expose qu'une porte ; la CI refuse tout import interdit. Le nouveau se branche à côté.
3. **Prévenir sans bloquer.** Décision et e-mail sont enregistrés ensemble ; un worker séparé envoie. Serveur mail en panne : la validation passe, l'e-mail part plus tard.

POUR NOS CLIENTS Leurs données restent chez eux, et le salarié est toujours prévenu.

3 · LA PREUVE

Les 4 failles critiques fermées

Chaque correction est tenue par un test qui échouerait si le défaut revenait.

FAILLE	CORRECTION	PREUVE (TEST)
Reçus d'une autre entreprise	Accès vérifié par le module, puis par la base	chaque rôle, autre entreprise → 404
Filet de la base désactivé	Rôle applicatif soumis à l'isolation	sans entreprise, rien n'est visible
Injection dans le tri	Liste blanche des colonnes	`?sort=` inconnu → tri par défaut
Secret dans le code	Secret obligatoire à l'environnement	sans secret, l'app refuse de démarrer

LIMITE ASSUMÉE L'OCR reste synchrone : le rendre asynchrone changerait l'API. Risque atténué (taille et concurrence plafonnées).

12/12

critiques et élevés corrigés · 29 défauts au total

10 → 0

parcours E2E en échec : l'app d'origine en ratait 10 sur 37, la nôtre passe **37/37**

620 tests · SonarQube : bugs + smells **7** → **0**, duplication **9,3** → **0 %**, couverture **66** → **75 %**

POUR NOS CLIENTS Mêmes parcours, même API, et des données enfin isolées entre entreprises.

4 · LA DÉMONSTRATION

Une fonctionnalité, un prompt

Chaque outil ferme un risque précis : l'agent intègre, il n'improvise pas.

1

AGENTS.md

Architecture, règles et commandes, chargés à chaque session.

Évite : du code posé au mauvais endroit

2

Skills

Recette TDD en 10 étapes : `ajouter-une-fonctionnalite`.

Évite : un test, le contrat d'API ou la doc oubliés

3

Revue

Sous-agent `revue-architecture` sur le diff.

Évite : une règle contournée qui compile quand même

4

Garde-fous

`npm run verify` : lint, types, depcruise, 620 tests.

Évite : un import interdit ou une régression

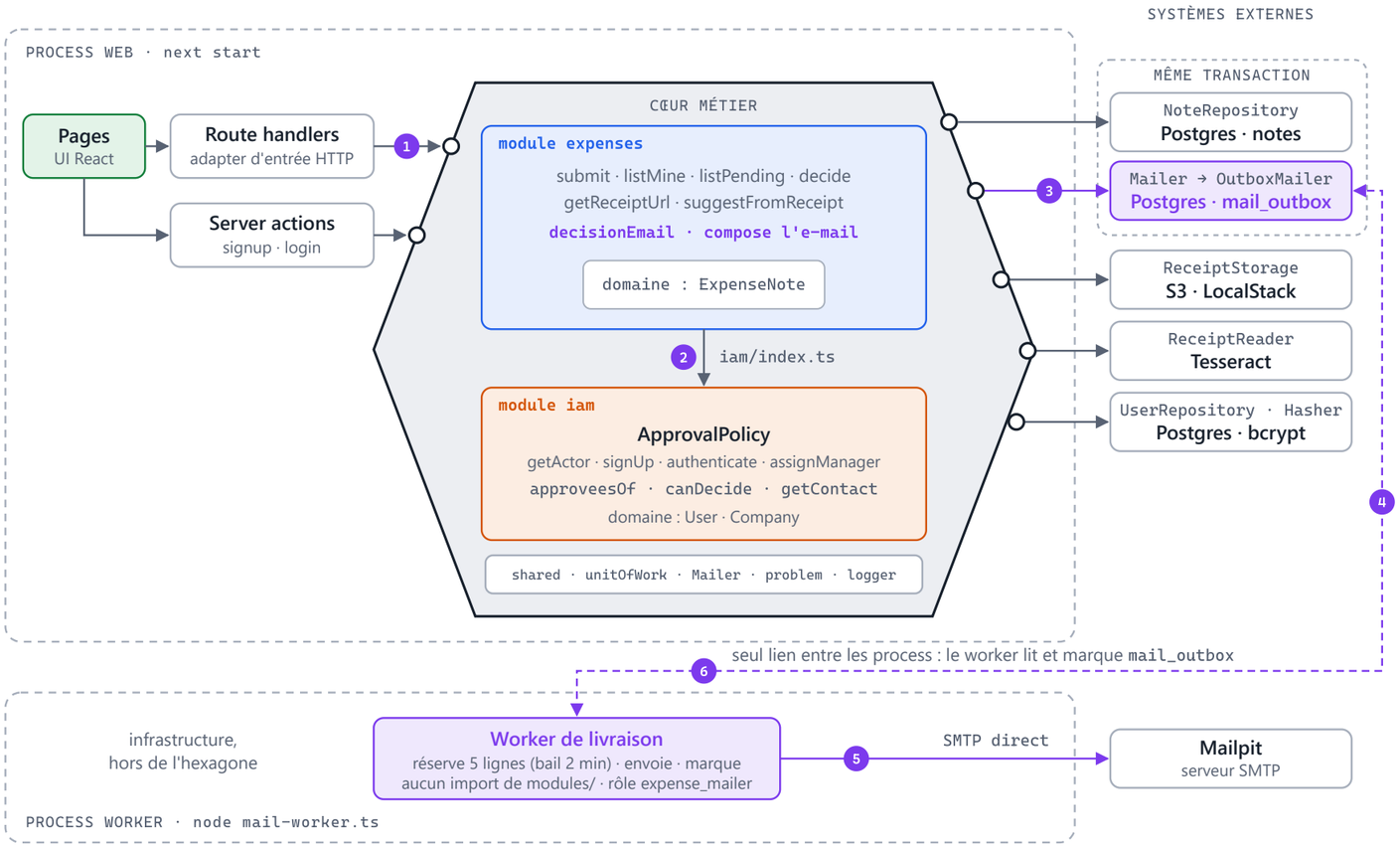
Lisible par l'humain aussi | ADR : pourquoi chaque choix | Conventions : où va chaque fichier | Runbooks et CHANGELOG du contrat

Répétition générale	≈ 12 min jusqu'au vert	0 intervention humaine	0 constat bloquant

MERCI Questions, puis live coding.

ANNEXE · ARCHITECTURE DÉTAILLÉE

Monolithe modulaire, hexagonal, avec outbox



1. Le manager valide une note : **l'API répond comme avant**.
2. « Qui peut valider » est écrit **à un seul endroit** ; deux validations simultanées, une seule gagne.
3. La décision et l'e-mail sont enregistrés **ensemble** : jamais l'un sans l'autre.
4. Un process séparé prend les e-mails en attente ; **s'il plante, il reprend** où il en était.
5. Une panne du serveur mail **ne bloque jamais** une validation.
6. Échec : nouvel essai, puis mise de côté. **Un doublon est possible, une perte jamais.**

À RETENIR Règles vérifiées en CI ; les deux process ne se parlent que via `mail_outbox` .